SALVADOR SANCHEZ ORTEGA

Programación 1ºDAW

BLOQUE 5

UNIDAD 10

21. ¿Qué significan las siglas JDBC? ¿Qué es JDBC? ¿Para qué sirve JDBC?

- **JDBC** = *Java DataBase Connectivity.*
- Es una **API de Java**.
- Sirve para que un programa Java pueda **conectarse y trabajar con bases de datos** (consultar, insertar, modificar, borrar datos).

22. ¿Cómo se instala JDBC?

- No se instala como tal ❌
- Ya viene incluido en Java (JDK).
- Lo único que necesitas es el **driver de la base de datos** (por ejemplo, MySQL Connector).

23. ¿Qué hay que hacer para poder usar JDBC en un proyecto de NetBeans?

- Añadir el **driver JDBC** (archivo `.jar`) al proyecto.
- Importar las librerías (`java.sql.*`).
- Crear la conexión con la base de datos.

24. Partes de la URL: `jdbc:mysql://localhost:3333/juanjo`

- `jdbc` → tipo de conexión
- `mysql` → gestor de base de datos
- `localhost` → servidor (tu propio PC)
- `3333` → puerto
- `juanjo` → nombre de la base de datos

25. ¿Qué operaciones debe realizar un programa para trabajar con BD?

1. Cargar el driver
2. Establecer conexión
3. Crear una consulta (SQL)
4. Ejecutar la consulta
5. Procesar resultados
6. Cerrar conexión

26. ¿Para qué sirven los objetos Statement?

- Sirven para **ejecutar consultas SQL** desde Java.

27. ¿Para qué sirven los objetos Connection?

- Representan la **conexión con la base de datos**.

28. ¿Cuándo se usan los objetos ResultSet?

- Cuando haces una consulta `SELECT`.
- Sirven para **recorrer y leer los resultados**.

29. ¿Qué hace `while(result1.next())`?

- Va recorriendo fila por fila el `ResultSet`.
- Cada vez que hace `next()`, pasa a la siguiente fila hasta que no haya más.

30. Diferencia entre `executeQuery` y `executeUpdate`

- `executeQuery()` → para **consultas SELECT** → devuelve `ResultSet`
- `executeUpdate()` → para **INSERT, UPDATE, DELETE** → devuelve número de filas afectadas

PREGUNTAS DE CÓDIGO

```
17      public static void main(String[] args) {
18          System.out.println(x: "Hello World!");
19          try{
20              Connection miConexion = DriverManager.getConnection(url: "jdbc:mysql://localhost:3333/juanj
21              Statement miConsulta = miConexion.createStatement();
22              ResultSet miRespuesta = miConsulta.executeQuery(string: "SELECT * FROM empleados");
23              double total=0;
24              System.out.println(x: "Cod\tNombre: \t\tFecha Nac.:\tSueldo: ");
25              while(miRespuesta.next()){
26                  System.out.println(miRespuesta.getInt(i: 1)+"\t"+miRespuesta.getString(i: 2)+"\t" +
27                          miRespuesta.getDate(i: 3) +"\t"+ miRespuesta.getDouble(i: 4));
28                  total+=miRespuesta.getDouble(i: 4);
29              }
30              System.out.println("el sueldo total es de "+total);
32          }catch(Exception e){
33              System.out.println("Error en Base de dAtos. "+e.getMessage());
34          }
        }
```

- `System.out.println("Hello World!");` → imprime mensaje por pantalla.
- `Connection miConexion = DriverManager.getConnection(...)` → crea la conexión con la BD.
- `Statement miConsulta = miConexion.createStatement();` → crea objeto para ejecutar SQL.
- `ResultSet miRespuesta = miConsulta.executeQuery("SELECT * FROM empleados");` → ejecuta consulta y guarda resultados.
- `double total=0;` → variable para acumular sueldos.
- `System.out.println(...)` → imprime cabecera.
- `while(miRespuesta.next())` → recorre cada fila de la tabla.
- `System.out.println(...)` → muestra datos (id, nombre, fecha, sueldo).
- `total += miRespuesta.getDouble(4);` → suma el sueldo al total.
- `System.out.println("el sueldo total es "+total);` → muestra suma total.
- `catch(Exception e)` → captura errores.
- `System.out.println(...)` → muestra mensaje de error.

UNIDAD 9

1. Diferencia JavaFX vs JavaFXML

- **JavaFX** 👉 interfaz gráfica creada **solo con código Java**.
- **JavaFXML** 👉 interfaz definida en **archivos FXML (XML)** + controlador en Java.

👉 Resumen:

- JavaFX = todo en código
- JavaFXML = separa diseño (FXML) y lógica (Java)

2. ¿Qué es SDK y para qué sirve?

- **SDK (Software Development Kit)**
  👉 Es un conjunto de herramientas para desarrollar aplicaciones.

En JavaFX:

- Incluye librerías, APIs, etc.
- Sirve para poder **usar JavaFX fuera del JDK**.

3. Pasos para instalar SDK (JavaFX)

1. Descargar JavaFX SDK desde la web oficial
2. Descomprimir el archivo
3. Guardarlo en una carpeta (ej: `/opt/javafx` o `C:\javafx`)

4. Pasos para usar SDK en una aplicación

1. Ir a propiedades del proyecto
2. Añadir librerías externas (JavaFX SDK → carpeta `lib`)
3. Configurar VM Options:

   --module-path /ruta/javafx/lib --add-modules javafx.controls,javafx.fxml

5. ¿Qué es Scene Builder y para qué sirve?

- Es una herramienta gráfica para diseñar interfaces JavaFX. 🎨
- Permite crear ventanas **arrastrando elementos (drag & drop)** sin programar.

6. Pasos para instalar Scene Builder

1. Descargar desde la web oficial
2. Instalar como cualquier programa
3. (Opcional) Conectarlo con NetBeans

7. Pasos para usar Scene Builder en una aplicación

1. Crear archivo `.fxml`
2. Abrirlo con Scene Builder
3. Diseñar la interfaz
4. Asociar un controlador Java
5. Usarlo desde el programa

9. Modelo-Vista-Controlador (MVC)

- **Modelo** → datos (BD, lógica)
- **Vista** → interfaz gráfica
- **Controlador** → conecta modelo y vista

👉 Ventajas:

- Código más organizado
- Fácil mantenimiento
- Reutilización
- Separación de responsabilidades

10. CRUD

- **C** → Create (crear)
- **R** → Read (leer)
- **U** → Update (actualizar)
- **D** → Delete (borrar)

👉 Aplicación CRUD:
Programa que permite **gestionar datos (crear, ver, modificar y eliminar)**.

11. ¿Por qué no crear JavaFXML directamente en NetBeans?

- NetBeans no tiene plantilla directa para JavaFXML
- Primero creas proyecto JavaFX
- Luego añades:
  - archivo `.fxml`
  - controlador
    👉 Porque FXML es una extensión de JavaFX, no un proyecto independiente.

12. Pasos para crear aplicación JavaFX

1. Crear proyecto Java
2. Añadir JavaFX SDK
3. Crear clase que extienda `Application`
4. Crear `Stage`, `Scene` y nodos
5. Ejecutar con `launch()`

13. Pasos para crear aplicación JavaFXML

1. Crear proyecto JavaFX
2. Crear archivo `.fxml`
3. Diseñar interfaz (Scene Builder)
4. Crear clase controlador
5. Enlazar FXML con controlador
6. Cargar FXML desde Java:

FXMLLoader.load(getClass().getResource("vista.fxml"));

27. ¿Cuál debe ser la extensión del fichero que contiene el siguiente código?
28. ¿Cuál es el papel de este fichero en MODELO - VISTA - CONTROLADOR?
29. ¿Quién crees que llama a la función **registrar**?
30. ¿Qué hace la línea 33?
31. ¿Para qué crees que sirve la función **registrar**?

```
17  public class ModeloDAO {
18      private ConexionMysql realizarConexion;
19
20
21      public ModeloDAO() {
22          this.realizarConexion = new ConexionMysql();
23      }
24
25      public boolean registrar(Modelo modelo){
26          try { String SQL="insert into empleados(nombre,fechaNac,sueldo) values(?,?,?)";
27          Connection conexion = this.realizarConexion.getConexion();
28          PreparedStatement sentencia = conexion.prepareStatement(string: SQL); //System.ou
29          sentencia.setString(i: 1,string: modelo.getNombre());
30          sentencia.setString(i: 2,string: modelo.getFecha().toString());
31          sentencia.setDouble(i: 3, d: modelo.getSueldo());  // System.out.println("%-$ "+s
32
33          sentencia.executeUpdate();
34          sentencia.close();
35          return true;
```

27. ¿Cuál debe ser la extensión del fichero?

👉 **`.java`**

Porque es una clase Java (`public class ModeloDAO`).

---

28. ¿Cuál es el papel de este fichero en MVC?

👉 Es el **MODELO**

- Se encarga de:
    - Acceder a la base de datos

    - Ejecutar consultas SQL
- No tiene interfaz ni interacción directa con el usuario

---

29. ¿Quién llama a la función `registrar`?

👉 El **CONTROLADOR**

- El controlador recibe eventos (por ejemplo, de la interfaz)
- Luego llama al modelo (`registrar`) para guardar datos

---

30. ¿Qué hace la línea 33?

👉

```
sentencia.close();
```

- Cierra el `PreparedStatement`
- Libera recursos de la base de datos

---

31. ¿Para qué sirve la función `registrar`?

👉 Sirve para **insertar datos en la base de datos**

- Ejecuta un `INSERT INTO empleados`
- Guarda un nuevo empleado
- Devuelve `true` si todo va bien